

Projeto Tipos de dados abstratos

Aluno: Gabriel Medeiros do Nascimento

Introdução

- forma de organizar dados e operações sem se preocupar, a princípio, com a implementação interna.
- definir **o que a estrutura faz**, e não **como ela funciona por dentro**.
- Exemplo:
 - conjunto.add (7) -> Adiciona o valor “7” ao conjunto.
- Vantagens:
 - organização do código
 - reutilização
 - facilidade de manutenção
 - • encapsulamento.

Descrição teórica

- Pilha (Stack)
 - o último elemento inserido é o primeiro a sair.
 - empilhar
 - desempilhar
 - acessar o topo
- Fila (Queue)
 - o primeiro elemento inserido é o primeiro removido.
 - enfileirar
 - desenfileirar
 - acessar o início
- Lista
 - Estrutura que armazena elementos em sequência, permitindo inserções e remoções em diferentes posições.
 - sequencial
 - encadeada
 - duplamente encadeada
- Conjunto
 - Estrutura que armazena elementos sem repetição.
 - união;
 - interseção;
 - diferença;

Trechos Importantes:

No programa de **Conjuntos**, como principal parte temos a definição de cada função no *.h*

```
class Conjunto{
private:
    int* dados;
    int tam;
    int tamMax = 100;

public:
    Conjunto();
    ~Conjunto();

    void add(int elem); // adiciona elemento
    void remove(int elem); // remove elemento apos procurar em todo o
conjunto.
    bool contains(int elem) const; //Verifica se o valor já está inserido
    Conjunto unionSet (const Conjunto& B); // união entre dois conjuntos
    Conjunto intersection (const Conjunto& B); // cria um novo conjunto
com a interseção entre dois conjuntos
    void print(); // imprime o conjunto
    int size(); //retorna o tamanho do conjunto
};
```

Por se tratar de um programa usado para aprendizado e em pequena escala, foi definido um valor máximo de elemento por dados do tipo *Conjunto*.

No *main.cpp* é usado um *while* para testar se o valor inserido é viável.

```
while(sizeArrayA>49){
    cout << "entrada invalida, inserir novo valor: ";
    cin >> sizeArrayA;
}
```

Já no programa **Sequencia**, temos:

```
private:
    int* dados;
    int tam;
    int tamMax = 100;

public:
    Sequencia();
    ~Sequencia();

    void insert(int pos, int elem); //insere um elemento de acordo com a
posicao desejada
    void remove(int pos); //remove um elemento da posição X
    int get(int pos); //retorna um elemento da posição X
    void print(); //imprime o conjunto
    int size(); //retorna o valor do conjunto

};
```

Dá mesma forma do programa anterior, foi definido um valor máximo pro *array*.

No *main.cpp*, foi criado um *array* de cópia.

```
Sequencia array;
Sequencia arrayCopy;
```

```
//cria um array copia
for(int i = 0; i < array.size(); i++){
    arrayCopy.insert(i,array.get(i));
}
```

Além disso mantendo a interação com o usuário de maneira mais intuitiva.

```
int posRemove;
cout << "Escolha posicao do numero que vai ser removido: ";
cin >> posRemove;
cout << endl;
arrayCopy.remove(posRemove-1);
arrayCopy.print();
cout << "Escolha posicao do segundo numero que vai ser removido: ";
cin >> posRemove;
cout << endl;
arrayCopy.remove(posRemove-1);
```

Análise de complexidade:

Operação	Complexidade	Motivo
insert(pos, elem)	$O(n)$	pode precisar deslocar elementos
remove(pos)	$O(n)$	desloca elementos após remoção
get(pos)	$O(1)$	acesso direto por índice
print()	$O(n)$	percorre todos elementos
add(elem)	$O(n)$	precisa verificar repetição (contains)
contains(elem)	$O(n)$	busca linear
unionSet()	$O(n^2)$	vários add/contains
intersection()	$O(n^2)$	percorre e busca no outro conjunto
pushBack(int elem)	$O(1)$	adiciona no final
pushFront(int elem)	$O(n)$	desloca todos elementos
find(int elem)	$O(n)$	mesma ideia do contains
size()	$O(1)$	retorna variável tam
print()	$O(n)$	percorre todos elementos

Conclusão:

O projeto permite compreender na prática o funcionamento de tipo abstrato de dados junto a estruturas comumente usadas para organização de elementos, ao invés de fazer o uso de bibliotecas do próprio c++, foi criado cada função como forma de melhor compreender o funcionamento interno dessas estruturas e o uso de classes e encapsulamentos.

Além disso reforçando o uso de conceitos fundamentais, como construtores, destrutores, passagem por referência e alocação dinâmica.